When Pair Programming Gets Emotional: A Case Study of What LLMs Still Miss

Rahat Rizvi Rahman
Virginia Commonwealth University, Richmond, VA, USA
Email: rahmanr12@vcu.edu

Seung Pyo Kang, Se Jin Jang, Seungmoo Lee, and Chungyean Cho
Korea National University of Arts, Seoul, South Korea
Email: pyoism@karts.ac.kr, notimpatient@karts.ac.kr, cinefact@gmail.com, yellowsub@karts.ac.kr

Kostadin Damevski
Virginia Commonwealth University, Richmond, VA, USA
Email: kdamevski@vcu.edu

Emotions play a key role in collaborative software practices like pair programming. In this case study, we analyze pair programming videos to assess whether LLMs can serve as pair programming partners. We adapt the Empathetic Dialogues benchmark to create scenarios that simulate how an LLM responds in emotional moments in pair programming. We find that LLMs generally match the level of empathy humans provide in emotional moments during pair programming. However, they sometimes get lost in conversational dynamics and miss the big picture, and prioritize task progression over discussion. Our findings highlight both the potential and current limitations of LLMs in emotionally sensitive programming contexts.

Keywords: Pair Programming, Human–AI Collaboration, Emotions in Software Engineering, Emotion Recognition, Empathetic Dialogues, Conversational Agents, Large Language Models (LLMs), Programming Intents, Programming Acts

1. Introduction

With the advent of powerful AI, LLMs in particular, researchers have started examining the use of AI for pair programming [1–3]. In this human–AI setup, the AI acts as the pair programming partner, most often taking the role of a driver who writes code or provides targeted improvement suggestions. Unlike popular uses of AI that focus solely on generating code from prompts, this envisions AI systems as active collaborators that engage as equal partners in the full programming process. Human–AI pair programming (i.e., *pAIr programming*) systems emphasize continuous, dialog-based interaction, where the AI contributes to reasoning, exploration, and shared decision-making throughout software development.

Emotions are central to effective collaboration in software development [4]. Positive emotions enhance social interactions and perceived productivity [5], while the collaborative context itself often triggers emotional responses that must be managed, particularly in pair-based work [6, 7]. This raises an important question: can AI systems provide the emotional intelligence required to be effective pair programming partners? Prior work has explored similar questions in the context of AI chatbots and programming tools [8]. While we do not yet have large-scale datasets of actual human–AI pair programming sessions, we can approximate such interactions by using real human–human conversations and projecting how an LLM might respond if it were a conversational peer in those moments. This approach allows us to explore, in a simulated setting, whether current LLMs are capable of producing responses that align with the emotional and task-related context of informal pair programming discussions

In other areas of software engineering, developers have already expressed a desire for more emotionally aware AI interactions. For example, in a study on AI programming assistants, a participant noted, *"I'd like to be able to ask it to calm down sometimes instead of constantly trying to suggest random stuff"* [9]. Meanwhile, findings from a medical domain study revealed that ChatGPT's responses were perceived as more empathetic and of higher quality than those provided by physicians [10]. These insights suggest a growing expectation for emotionally intelligent behavior in AI systems.

In this paper, we investigate how emotions shape pair programming interactions and examine the extent to which large language models can recognize and respond to these emotions in collaborative coding contexts. First, we analyze real pair programming sessions to uncover when and how emotions such as confusion, curiosity, and approval emerge across different conversational intents and programming activities. Next, using these insights, we simulate human–AI pair programming scenarios to assess how well current LLMs respond during emotionally charged moments, comparing their reactions to those of human partners.

To conduct this analysis, we conduct a case study based on emotional points in three pair programming videos from YouTube (ranging from 38 minutes to 1 hour and 34 minutes) that have been used in prior pair programming studies. The videos reflect realistic, representative pair programming interactions consisting of a driver and a navigator working on the solution to a technical problem. Our analysis of the videos highlights the moments when emotions surface in pair programming and how they shape the flow of collaboration. For instance, we observed that different emotions arise depending on the conversational intent and programming activity, e.g., writing source code often leads to approval, while code comprehension is often related to confusion.

Next, we randomly select emotional points representing different types of emotions from the videos and reframe them using a well-known LLM emotional awareness benchmark, EmpatheticDialogues [11]. This projection does not capture the

full dynamics of real human–AI interaction, but instead serves as an initial probe into whether current LLMs can produce emotionally and contextually appropriate utterances in informal, collaborative programming exchanges. Based on these data instances, we evaluate how LLMs respond in the corresponding scenarios. Our results show that LLMs generally match the level of empathy expressed by human peers in most cases, but tend to focus more on task progression than humans, i.e., LLMs encourage a more rapid pace of work.

2. Methodology

To facilitate our case study of emotions in pair programming, we selected three pair programming YouTube videos, previously studied by Hart et al. [12]. These videos are relatively lengthy and represent a realistic pair programming session on a variety of different technical topics:

(1) *First Time Pair Programming on ApprovalTests.Java* – `https://www.youtube.com/watch?v=FuCLXRd71oo`; duration 1h 34m 47s
(2) *Pair Programming a Facebook Messenger Bot* – `https://www.youtube.com/watch?v=zFO1cRr5-qY`; duration 57m 43s
(3) *Building a Tower Defense Game in REACT* – `https://www.youtube.com/watch?v=mdVPc5n9-VQ`; duration 38m 14s

To extract high quality emotional expressions in these videos, we developed a multimodal pipeline that integrates: 1) speech recognition, 2) automated (text and sound) emotion recognition, and 3) manual validation. This design follows prior multimodal emotion recognition research showing that modeling acoustic and textual cues separately and integrating their outputs can preserve modality-specific characteristics while benefiting from complementary information across modalities [13].

Speech Recognition. First, using the PyAnnotate [14] toolkit, we extracted the start and end times of each utterance. Then we manually labeled the corresponding speaker based on watching the video. The timestamps were used to generate transcriptions using OpenAI's Whisper-medium.en model [15]. Finally, one of the authors manually verified and corrected any errors in the transcription through careful inspection of the video recordings.

Automated Emotion Recognition. Next, we performed text-based emotion recognition using the *Llama-3.3-70B Instruct (Q3_K_M quantized)* model. To improve coherence and address incomplete transcriptions, we merged utterances spoken by the same speaker within a 3-second interval. We tested different merging thresholds (2–5 s). A 2-s window resulted segments too short for reliable emotion detection, while windows longer than 3 s produced overly long segments with fewer emotional instances. The 3-s threshold offered the best balance between context and granularity. This merging applied only to consecutive utterances from the same

speaker, preserving each speaker's natural emotional flow without mixing distinct emotions. We then prompted the LLM to extract all available emotion labels and their corresponding intensity scores in the combined utterances. We used a prompt that briefly described the task and requested a JSON output containing the predicted emotions and associated confidence levels. The target emotion set consisted of the 27 emotion categories defined in the GoEmotions dataset [16], which specfically targets text emotion detection. To ensure a strong signal, we only retained emotions predicted with intensity scores of 0.6 or higher as it was the median intensity observed across the analyzed videos.

For sound-based emotion recognition (SER), we used a pretrained acoustic model by Osman et al. [17] The SER model predicted basic emotions, including Anger, Disgust, Fear, Happiness, Neutral, Sadness, and Surprise, based on prosodic and spectral features extracted from each utterance. We filtered out low-confidence predictions and retained only those with intensity scores of 0.21 or higher. This cutoff corresponded to the median SER intensity values across the three videos (0.219, 0.234, and 0.249), ensuring that only reliable predictions were used to augment the emotion dataset

Manual Validation Both the sound-based and text-based emotion predictions were validated by two expert annotators, each with an undergraduate degree in computer science and prior experience annotating software engineering communication data. The inter-annotator agreement, measured using Cohen's kappa (κ) over the combined annotations, excluding Neutral, which was often used as a default category, was 0.27, which falls within the fair range according to Landis and Koch [18]. While this indicates agreement above chance, it also reflects relatively low consistency between annotators, highlighting the inherent difficulty of reliably identifying emotions in pair programming transcripts. Similar levels of agreement have been reported in prior affective computing studies involving real-world speech data [7, 18]. Overall, the annotators agreed on 63.07% of all labeled emotions.

The majority of disagreements, approximately 470 cases, or 83.63% of all disagreements, occurred in the annotation of seven emotions: Confusion, Approval, Curiosity, Optimism, Excitement, Happiness, and Relief. Among these, Annotator 2 labeled 60% more instances as valid than Annotator 1 for Confusion, Curiosity, Optimism, Excitement, and Relief. Conversely, Annotator 1 marked 24% more instances of Approval as valid than Annotator 2. For Happiness, all 48 disagreed cases involved sound-based predictions, whereas 97.66% of the remaining disagreements involved text-based predictions. These patterns suggest that both the modality of the prediction (sound vs. text) and individual annotator bias significantly influenced the annotation process. To resolve the disagreements, a third, more experienced annotator reviewed and resolved all disputed labels, resulting in the finalized version of the validated dataset. The third annotator re-evaluated each disputed instance by reviewing the corresponding video segment and transcript context before assigning the final label.

Table 1. Emotion-wise Cohen's kappa (κ) for annotator agreement on the combined dataset (three videos), excluding *Neutral*. Only emotions with at least 20 candidate instances ($n \geq 20$) are shown.

Emotion	n	κ(A1–A2)	κ(A1–GPT4o)	κ(A2–GPT4o)
Anger	52	0.64	0.46	0.39
Curiosity	189	0.32	0.28	0.17
Happiness	164	0.31	0.31	0.22
Excitement	114	0.20	0.32	0.32
Approval	227	0.19	0.06	0.18
Confusion	223	0.19	0.07	0.10
Relief	91	0.21	0.11	0.13
Disappointment	28	0.27	0.15	0.08
Realization	37	0.27	0.37	0.28
Optimism	106	0.06	0.15	0.10
Amusement	43	0.07	0.18	0.00
Annoyance	37	0.41	0.01	0.00
Disapproval	31	0.09	-0.03	0.30
Fear	26	0.00	0.00	—
Sadness	42	-0.04	0.00	0.00

A1 = Annotator 1, A2 = Annotator 2. "—" indicates undefined κ due to degenerate label distributions.

GPT-4o Validation Setup To further examine annotation reliability, we introduced GPT-4o as an additional independent annotator. GPT-4o received only textual transcripts and short conversational context, mirroring the information used by human annotators for text-based validation. For each utterance we provided: (1) the target utterance with speaker identifier, (2) a window of preceding utterances, and (3) candidate emotion labels predicted by the automated classifiers. GPT-4o returned a structured JSON response indicating whether each candidate emotion was plausible (1) or not plausible (0). This allowed us to treat GPT-4o as an additional rater performing the same binary validation task as the human annotators.

Agreement Patterns with GPT-4o Across all three videos combined, GPT-4o reaches a level of agreement with each human annotator comparable to the agreement between the two humans. Excluding *Neutral*, human–human agreement falls in the "fair" range ($\kappa \approx 0.27$), and agreement between GPT-4o and each annotator is of similar magnitude. This suggests that even a strong general-purpose language model does not achieve substantially higher consistency than human annotators on this task.

Table 1 summarizes emotion-wise inter-annotator agreement for the pooled dataset across all three videos, excluding *Neutral*. The table reports Cohen's kappa between the two human annotators and between each annotator and GPT-4o.

Examining agreement at the level of individual emotions reveals clear differences in annotation difficulty. Using the pooled dataset across all three videos (excluding *Neutral*), emotions such as *Anger*, *Curiosity*, *Happiness*, *Excitement*, *Relief*, *Dis-*

appointment, and *Realization* exhibit moderate human–human agreement (κ up to approximately 0.6) and non-trivial agreement with GPT-4o, suggesting relatively clear contextual cues.

In contrast, emotions such as *Sadness*, *Fear*, and several low-frequency labels show κ values close to zero or negative between annotators and similarly low agreement with GPT-4o. Even frequently occurring labels such as *Confusion*, *Approval*, and *Optimism*, which account for many disagreements, reach only fair agreement across raters.

We also observe systematic differences across annotators and modalities. As noted earlier, annotators differ in which emotions they tend to validate. For example, *Disapproval* ($n = 31$) shows low human–human agreement ($\kappa = 0.09$), while GPT-4o aligns more closely with one annotator than the other (Table 1).

Separating candidate labels by source modality further reveals that some emotions are easier to detect in specific modalities. For instance, in the sound-only subset *Anger* shows relatively strong agreement across raters, whereas *Sadness* remains difficult to annotate reliably even with acoustic cues.

Interpreting Low Agreement The observed $\kappa \approx 0.27$ reflects the inherent subjectivity of emotion interpretation in spontaneous pair programming dialogue rather than annotation noise. Such conversations contain short, context-dependent utterances where emotional signals are subtle and intertwined with technical reasoning. Under these conditions, even a powerful language model with consistent instructions agrees with human annotators only at a fair level and shares many of the same ambiguities. These results suggest that modest agreement levels are an intrinsic property of the task.

Table 2. Examples of Emotional Utterances Across Pair Programming Conversational Intents and Programming Acts.

Intent	**Example Utterance**
Answer	*Yeah, because it got changed in the readme...*
Feedback	*Yeah, that's actually not a bad idea. Let me check really quickly...*
Task Related	*What do you think about the idea of setting up GitHub actions so that the project is built?*
Task Control	*Let's go from there. So firstly run me through what it is you're trying to do here...*
Think Aloud	*So it is. Could it be that it never...*
Prog. Act	**Example Utterance**
Attend Location	*Show me that we have it in the right place again. Can you go to the...*
Code	*Alright sweet, get rid of that extra space.*
Comprehension	*It's building the artifacts, I think, but it's not installing them.*
Idea	*We can even set up a matrix there that it runs on Java 8, 11, and 14, for example.*
Run Code/Test	*And now if we click on actions, we should be... I skewed so it should start in a second.*
Task Goal	*Can you update your .NET?*

3. Emotions in Pair Programming

To relate the annotated emotions to pair programming tasks, we adopted the hierarchical classification scheme proposed by Robe et al. [19], which categorizes pair pro-

gramming utterances along dimensions of task *intent* and associated *programming act*. Specifically, one of the paper's authors annotated each utterance, while a second author reviewed the annotation. The two discussed and resolved disagreements in person. An *intent* refers to the underlying communicative purpose of the utterance (e.g., asking a question, providing feedback), while *programming act* describes the type of programming activity being performed or discussed. Programming act labels were applied only when the utterance was directly related to a programming activity, so not on every utterance. In addition, we labeled the *role* of the speaker in each utterance as either *Navigator* or *Driver*, determined by observing both the video recording and the audio to identify the participant's active role at that moment.

The intent categories proposed by Robe et al. include: *Task Control*, *Task Related*, *Answer*, *Feedback*, *Greeting*, *Think Aloud*, *Thanking*, *Repeat*, and *Uncertain*. For programming acts, we adopted the following labels: *Task Goal*, *Comprehension*, *Attend Location*, *Code*, *Idea*, and *Run Code/Test*. Table 2 shows examples of utterances from our dataset for different intents and programming acts.

Table 3. Emotion frequency and distribution percentage (in brackets) across Conversational Intents and Programming Acts

Intent	**Approval**	**Confusion**	**Curiosity**	**Excitement**	**Happiness**	**Optimism**	**Realization**	**Relief**
Answer	47 (**70.1%**)	7 (10.4%)	4 (6.0%)	1 (1.5%)	1 (1.5%)	2 (3.0%)	3 (4.5%)	2 (3.0%)
Feedback	28 (**33.7%**)	3 (3.6%)	6 (7.2%)	11 (13.3%)	10 (12.0%)	10 (12.0%)	2 (2.4%)	13 (15.7%)
Task Related	40 (18.6%)	41 (19.1%)	58 (**27.0%**)	17 (7.9%)	21 (9.8%)	14 (6.5%)	17 (7.9%)	7 (3.3%)
Think Aloud	3 (6.5%)	15 (**32.6%**)	12 (26.1%)	4 (8.7%)	6 (13.0%)	1 (2.2%)	3 (6.5%)	2 (4.3%)
Uncertain	1 (1.2%)	50 (**61.0%**)	29 (35.4%)	1 (1.2%)	1 (1.2%)	0 (0.0%)	0 (0.0%)	0 (0.0%)
Prog. Act	—	—	—	—	—	—	—	—
Attend Location	8 (15.4%)	15 (28.8%)	23 (**44.2%**)	0 (0.0%)	3 (5.8%)	1 (1.9%)	2 (3.8%)	-
Code	28 (**49.1%**)	8 (14.0%)	12 (21.1%)	5 (8.8%)	2 (3.5%)	2 (3.5%)	0 (0.0%)	-
Comprehension	11 (9.6%)	46 (**40.0%**)	25 (21.7%)	5 (4.3%)	8 (7.0%)	3 (2.6%)	17 (14.8%)	-
Idea	7 (19.4%)	6 (16.7%)	13 (**36.1%**)	1 (2.8%)	5 (13.9%)	2 (5.6%)	2 (5.6%)	-
Run Code/Test	7 (13.7%)	4 (7.8%)	17 (**33.3%**)	12 (23.5%)	4 (7.8%)	6 (11.8%)	1 (2.0%)	-
Task Goal	11 (15.1%)	11 (15.1%)	16 (**21.9%**)	12 (16.4%)	12 (16.4%)	10 (13.7%)	1 (1.4%)	-

3.1. *Emotion Patterns Across Intents*

To understand how different types of interaction impacts emotions in pair programming, we analyzed the emotions associated with conversational intents and programming acts. Table 3 shows the emotion distributions across intents and programming acts, respectively. To focus on emotions that occurred with meaningful frequency, we included only those emotions that appeared at least ten times across all intents and programming acts in our dataset. The same threshold was applied to filter intents and programming acts based on their frequency across emotions.

Certain intents display clear supportive and encouraging emotional patterns. For instance, the Answer intent evoked *Approval* in 47 utterances, which is the domi-

nant emotion 70.1% of the time. This indicates that responses often contain affirmative feedback. Similarly, the Feedback intent showed *Approval* (33.7%), *Excitement* (13.3%), *Happiness* (12.0%), *Optimism* (12.0%), and *Relief* (15.7%), combining for 72 utterances, which is 86.7% of the cases.

In contrast, the Task Related intent, where people discuss the programming task, mostly shows *Curiosity* (58 utterances, 27.0%). The high frequency of *Curiosity* suggests that participants are often questioning code-related details. Interestingly, the Think Aloud statements, where programmers narrate what they are doing, were not emotionally neutral. It revealed *Confusion* (15 utterances, 32.6%) and *Curiosity* (12, 26.1%) most of the time. We see the same pattern in the *Uncertain* intent, where *Confusion* (50 utterances, 61.0%) and *Curiosity* (29, 35.4%) jointly cover most of the cases.

These findings reveal that pair programming intents involving technical coordination and response, such as Answer and Feedback, majorly invokes positive emotions like *Approval*. However, intents related to verbally narrating thoughts, task-related discussions, and uncertain utterances often convey *Confusion* and *Curiosity*, reflecting the complexity and cognitive efforts during these tasks. These observations imply that future emotionally aware AI pair-programming partners could benefit from adapting their responses to the intent type. For example, providing clarification during confusion or affirmation during feedback.

3.2. *Emotion Patterns Across Programming Acts*

Emotion distributions across programming acts offer additional insights. The Attend Location act, related to checking a specific line or part of code, exhibited *Curiosity* in 23 utterances (44.2%) and *Confusion* in 15 (28.8%), suggesting that these moments often reflect active investigation or uncertainty rather than simple code navigation. During the Code act, where code change-related discussions happen, an unexpected emotion *Approval* was observed 49.1% (28 utterances) of the time. This may indicate that collaborators tend to validate each other when discussing code-level changes. Other emotions *Curiosity* (21.1%), Confusion (14%), and *Excitement* (8.8%) were also frequently observed, suggesting emotional investment at the implementation level.

The Comprehension act, related to understanding implementation logic, revealed a strong relation toward *Confusion* (46 utterances, 40%) and *Curiosity* (25, 21.7%) emotions, indicating the challenge of interpreting unfamiliar logic. The Idea act, related to proposing new solutions, triggered *Curiosity* (13 utterances, 36.1%), *Approval* (7, 19.4%), and *Confusion* (6, 16.7%), suggesting that proposing solutions is both exciting and challenging. Similarly, Run Code/Test was associated with *Curiosity* (17 utterances, 33.3%) and *Excitement* (12, 23.5%), reflecting emotional anticipation tied to execution outcomes.

Overall, programming acts elicited a diverse set of emotional responses. Acts involving code exploration or code execution, such as Attend Location, Compre-

hension, and Idea, frequently showed *Curiosity* and *Confusion*, suggesting mental effort and uncertainty. In contrast, acts related to code modification, planning, and testing, such as Code, Task Goal, and Run Code/Test act, more often evoked positive emotions such as *Approval* and *Excitement*. These findings indicate that AI programming partners could adapt their behavior to the programming act. For example, supporting comprehension or idea-generation phases with more guidance and maintaining momentum or positive reinforcement during coding and testing activities.

4. LLM Responses to Emotions in Pair Programming

Next, we set out to understand how effectively LLMs can serve as pair programming assistants in the more emotionally charged moments present during these sessions.

4.1. *Data and Method*

We ground our LLM experiment in standard datasets for evaluating emotional capacity in these models. Recent work on empathetic dialogue systems explores how conversational agents can recognize and respond to emotional cues in user interactions [11, 20] Specifically, we found the EmpatheticDialogues benchmark to be relevant to our experimental design [11]. In this benchmark, each data instance begins with a situation (e.g., I've been hearing noises around the house at night) associated with an emotion label (e.g., Afraid). The speaker initiates a conversation based on the situation, followed by 4–8 utterances exchanged with a listener. The LLM is then asked to produce the next utterance in the role of the listener, which is evaluated for empathy in response to the speaker's emotional context.

Inspired by the EmpatheticDialogues benchmark, we randomly sampled three conversations for each of the eight emotions commonly observed in our dataset of pair programming videos. For this analysis, we constructed 26 emotional conversation cases. For each of these cases, we provided a brief 3–4 sentence description of the situation, the contents of the IDE editor and command window if a command was executed (e.g., output of running the project or a test), along with the previous 10 utterances from the conversation. We included more utterances than the original EmpatheticDialogues benchmark due to the generally shorter length of utterances in our data. Based on this input, we prompted the LLM to generate the next utterance, instructing it to respond as a peer programmer. We used open source *Llama3.3:70b-Instruct (Q3_K_M quantized)* model as it is a popular modern LLM that is representative of the current generation of such models. In addition to Llama3.3:70b-Instruct, we also generated comparison responses using GPT-4o using same setup. This cross-model replication allows us to examine whether our observations about empathy, conversational pacing, and failure cases are specific to the Llama architecture or generalize to a state-of-the-art model with a different design. We analyzed the LLM-generated responses qualitatively, compared to the human responses. We focusing on their empathy and relevance to the task. More

Table 4. Example of LLM Conversation Continuation.

Emotion	**Pair Programming Conversation**	**Human Next Utterance**	**Llama Next Utterance**
Approval	*Situation:* Two pair programmers collaborate on cleaning up open pull requests and enhancing the health and documentation of the ApprovalTests.Java project. Now, the navigator is guiding the driver through quickly creating a Maven project with unit testing. They wrote a minimal test using ApprovalTests and ran it. *Source Code:* <current editor and IDE command output> *Transcript:* Driver: Okay. OK. I am corrected. Driver: Ah, that created the whole file, okay. Navigator: You should not be happy with the J unit. We have a J in it there. Driver: Let's see, refresh. Driver: There we go. Navigator: Yeah and just verify, foo. Navigator: Yeah. And let's see that. work. Navigator: I think that just popped up a thing. Driver: Yeah. Yeah, yes. Driver: Looking good.	Navigator: Cool. Alright, so it does handle 1.6, which is good. I was expecting that. You can't build it in 1.6 obviously, but I think if you get the jar file... Jar, I am personally flabbergasted on why all builds are not completely backwards compatible.	Navigator: Great! Now, let's modify the test to verify a more complex output. How about we try with a multi-line string?

specifically, two authors independently labeled each response, then met in person to consolidate the labels using a card sorting process [21]. Each author compared the human and LLM responses and labeled which one appeared more empathetic or whether both showed a similar level of empathy. We used a practical, conversational definition of empathy: a response was considered empathetic if it acknowledged or appropriately reacted to the speaker's emotion. This approach follows prior conversational AI work [11] and was sufficient for our qualitative analysis. Table 4 presents an example pair programming conversation exhibiting Approval where the navigator speaks next, along with both the human and LLM responses.

4.2. *Key Findings*

We summarize the key findings in our analysis around a few themes:

1. LLMs are just as empathetic as humans in pair programming conversations. This pattern is consistent across two different model families. Across 26 conversations, we compared the human next utterance to responses from Llama3.3-70B-Instruct and GPT-4o using the same emotional scenarios and prompts. GPT-4o matched the human partner's empathy level in 25 cases and was more empathetic in 1 case, while Llama showed a nearly symmetric pattern with 24 cases of similar empathy, 1 favoring the human, and 1 favoring Llama. In 15 of the 24 cases with similar empathy levels, neither the human nor the LLMs acknowledged the emotional content of the utterance, instead responding only to its technical aspects. Overall, both models exhibited empathy to a similar degree as humans when responding to positive emotional cues (e.g., optimism, excitement, or approval), while all three, the humans and both models, were less responsive to subtler negative emotions such as confusion.

2. LLMs proposed a more rapid pace to the work than humans. We observed a notable difference in how the conversations progressed. Humans often paused to acknowledge a fact or to reflect on the pros and cons of a previous solution. In contrast, the LLMs frequently proposed a next step or suggested a course of action, thereby accelerating the pace. This occurred in 7 out of 26 conversations we examined. The observation is consistent with prior findings by Liang et al. [9], who noted that LLMs tend to be more focused in driving task completion. This can also be observed in the example in Table 4.

3. Despite the scenario and utterances, LLMs sometimes got lost in the conversation. Even with detailed input we provided the LLM, including the development scenario, source code, and a 10-utterance transcript, we found 6/26 cases where both the LLMs failed to address the most important aspects of the conversational flow. In these instances, the LLMs appeared to focus too narrowly on the immediate conversation, overlooking broader contextual cues that were more relevant to an effective response. This supports a recent finding by Laban et al. [22], who showed that LLMs often get lost in multi-turn conversations by focusing too much on recent turns and ignoring important context from earlier in the dialogue.

5. Threats to Validity

To support internal validity, we selected three publicly available pair programming YouTube videos previously used in research [12]. These videos reflect realistic programming tasks and interactions, but our findings are limited by the small dataset. With only three videos, generalization remains limited. In addition, the LLM response analysis is based on 26 emotional conversation cases constructed from the selected videos. This relatively small sample size limits the ability to draw strong statistical conclusions about LLM behavior. Future work should expand the dataset to increase coverage across emotions, roles, and programming contexts.

While results may differ across LLM architectures or fine-tuned variants, we mitigated this risk by selecting a widely used, instruction-tuned open-weight model (Llama3.3-70B-Instruct). To further examine potential architectural bias, we replicated our next-utterance experiment with the same setup using GPT-4o, a proprietary LLM from a different model family. GPT-4o produced responses with similar empathy and task relevance patterns to those of Llama: in most scenarios, its empathy was on par with the human partner, and it shared the same tendency to focus on task progression and to miss some nuanced emotional cues. These cross-model similarities suggest that our main qualitative findings about LLM behavior in emotional pair programming moments are not artifacts of a single architecture, but instead reflect broader tendencies in contemporary large language models.

We used the same LLM family (Llama-3.3-70B-Instruct) for both emotion annotation and empathy-response simulation, applied separately on the same transcribed sessions. The annotation results were not reused or fed into the second task. This setup ensured interpretive consistency while keeping the analyses independent,

serving as a controlled baseline for exploratory analysis.

The primary external threat lies in ecological validity. The analyzed sessions come from informal YouTube videos, not professional development settings. These sessions may differ from workplace pair programming in emotional tone, task complexity, or social dynamics. However, we selected these videos to capture natural, unscripted pair programming interactions that reflect genuine emotional exchanges while providing ethically accessible, realistic data without participant recruitment. Replication with data from industrial contexts or educational settings would strengthen generalizability.

6. Conclusions

In this paper, we analyzed the emotions that appears in pair programming session and examined how an LLM responds to emotionally charged moments. By studying three publicly available human–human pair programming videos, we identified patterns linking specific emotions to conversational intents and programming acts. Positive emotions such as *Approval* and *Excitement* were often observed in collaborative and feedback-related interactions. On the other hand, *Confusion* and *Curiosity* frequently experienced during comprehension, idea generation, and uncertain utterances. We extended this analysis by adapting EmpatheticDialogues benchmark to evaluate how LLM responses in comparable scenarios. Our preliminary results show that LLM can match human partners in empathy level, but they prioritize faster and immediate task resolution and sometimes get lost in the conversation. We observed these patterns consistently for both an open-weight model (Llama3.3-70B-Instruct) and a proprietary model (GPT-4o), indicating that these behaviors are likely common across current LLM architectures rather than unique to a single system.

7. Future Work

Future work should expand to a larger and more diverse dataset of pair programming sessions to better capture emotional and technical interactions across different task conditions. This would enable comparisons of emotional patterns and interaction styles across programming contexts, such as educational environments. It is also important to evaluate emotional and collaborative dynamics using various LLM architectures, including open-weight, proprietary, and fine-tuned models. Such comparisons can reveal differences in empathy, emotion recognition, and contextual awareness. Future studies should explore adaptive strategies for AI partners that adjust behavior based on the user's emotional state and task stage, balancing efficiency with supportive interaction. Finally, standardized benchmarks are needed to measure emotional responsiveness and conversational awareness in programming-focused AI systems, enabling consistent evaluation and progress toward emotionally intelligent AI partners.

Acknowledgments

This research was supported by Culture, Sports and Tourism RD Program through the Korea Creative Content Agency (KOCCA) grant funded by the Ministry of Culture, Sports and Tourism(MCST) in 2024 (Project Name: Empowering the Next Generation of Cultural Technology Specialists through Extended Reality (XR) Content with Generative AI and Emotionally Responsive AI Agents Project, Number: RS-2024-00393915, Contribution Rate: 100%).

References

[1] S. Imai, Is GitHub copilot a substitute for human pair-programming? an empirical study, in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, *ICSE '22*, (Association for Computing Machinery, New York, NY, USA, October 2022), pp. 319–321.

[2] A. Moradi Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M. C. Desmarais and Z. M. J. Jiang, GitHub Copilot AI pair programmer: Asset or Liability?, *Journal of Systems and Software* **203** (September 2023) p. 111734.

[3] S. Barke, M. B. James and N. Polikarpova, Grounded copilot: How programmers interact with code-generating models, *Proceedings of the ACM on Programming Languages* **7**(OOPSLA1) (2023) 85–111.

[4] D. Graziotin, X. Wang and P. Abrahamsson, Happy software developers solve problems better: psychological measurements in empirical software engineering, *PeerJ* **2** (2014) p. e289.

[5] D. Graziotin, F. Fagerholm, X. Wang and P. Abrahamsson, What happens when software developers are (un) happy, *Journal of Systems and Software* **140** (2018) 32–47.

[6] N. Novielli, F. Calefato and F. Lanubile, The challenges of sentiment detection in the social programmer ecosystem, in *Proceedings of the 7th international workshop on social software engineering*, 2015, pp. 33–40.

[7] D. Girardi, F. Lanubile, N. Novielli and A. Serebrenik, Emotions and Perceived Productivity of Software Developers at the Workplace, *IEEE Transactions on Software Engineering* **48** (September 2022) 3326–3341.

[8] A. Alami and N. A. Ernst, Human and machine: How software engineers perceive and engage with ai-assisted code reviews compared to their peers, *arXiv preprint arXiv:2501.02092* (2025).

[9] J. T. Liang, C. Yang and B. A. Myers, A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges, in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, (ACM, Lisbon Portugal, February 2024), pp. 1–13.

[10] J. W. Ayers, A. Poliak, M. Dredze, E. C. Leas, Z. Zhu, J. B. Kelley, D. J. Faix, A. M. Goodman, C. A. Longhurst, M. Hogarth and D. M. Smith, Comparing Physician and Artificial Intelligence Chatbot Responses to Patient Questions Posted to a Public Social Media Forum, *JAMA internal medicine* **183** (June 2023) 589–596.

[11] H. Rashkin, E. M. Smith, M. Li and Y.-L. Boureau, Towards empathetic open-domain conversation models: A new benchmark and dataset, in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, eds. A. Korhonen, D. Traum and L. Màrquez (Association for Computational Linguistics, Florence, Italy, July 2019), pp. 5370–5381.

[12] J. Hart, J. AuBuchon and S. K. Kuttal, Feasibility of using youtube conversations for

pair programming intent classification, in *2022 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, 2022, pp. 1–4.

[13] B. Pan, K. Hirota, Z. Jia and Y. Dai, A review of multimodal emotion recognition from datasets, preprocessing, features, and fusion methods, *Neurocomputing* **561** (2023) p. 126866.

[14] H. Bredin, pyannote.audio 2.1 speaker diarization pipeline: principle, benchmark, and recipe, in *Proc. INTERSPEECH 2023*, 2023.

[15] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey and I. Sutskever, Robust speech recognition via large-scale weak supervision (2022).

[16] D. Demszky, D. Movshovitz-Attias, J. Ko, A. Cowen, G. Nemade and S. Ravi, Goemotions: A dataset of fine-grained emotions, in *Proceedings of the 58th annual meeting of the association for computational linguistics*, 2020, pp. 4040–4054.

[17] M. Osman, D. Kaplan and T. Nadeem, SER Evals: In-domain and Out-of-domain Benchmarking for Speech Emotion Recognition, in *Proceedings of Interspeech 2024*, 2024, pp. 1395–1399.

[18] J. R. Landis and G. G. Koch, The measurement of observer agreement for categorical data, *biometrics* (1977) 159–174.

[19] P. Robe, S. K. Kuttal, J. AuBuchon and J. Hart, Pair programming conversations with agents vs. developers: challenges and opportunities for SE community, in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, *ESEC/FSE 2022*, (Association for Computing Machinery, New York, NY, USA, November 2022), pp. 319–331.

[20] A. Sharma, A. Miner, D. Atkins and T. Althoff, A computational approach to understanding empathy expressed in text-based mental health support, in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 5263–5276.

[21] D. Spencer, *Card sorting: Designing usable categories* (Rosenfeld Media, 2009).

[22] P. Laban, H. Hayashi, Y. Zhou and J. Neville, Llms get lost in multi-turn conversation, *arXiv preprint arXiv:2505.06120* (2025).